



PROPUESTA DE AUTOMATIZACIÓN DEL CICLO DE DESARROLLO

Desarrollo Asistido por IA

Convertir la fase de desarrollo de una task — desde que el equipo la recibe hasta dejar el PR abierto en GitHub — en un proceso ejecutable por IA: verificable, auditable y repetible.

Viva Aerobus

Proyecto Generic API

Plan de trabajo — 16 semanas

DOCUMENTO CONFIDENCIAL. ELABORADO POR TECH AND SOLVE PARA VIVA AEROBUS.

CONTENIDO

1. Resumen ejecutivo
2. El problema que resolvemos
3. Qué queremos lograr
4. El flujo de trabajo completo, paso a paso
5. Las fases, y por qué en ese orden
6. Detalle de cada fase y estimación
7. Cronograma y esfuerzo
8. Cómo sabremos que funcionó
9. Lo que sigue después
10. Requerimientos para hacerlo realidad
11. Cambios que requiere la dinámica de trabajo
12. Riesgos y mitigación

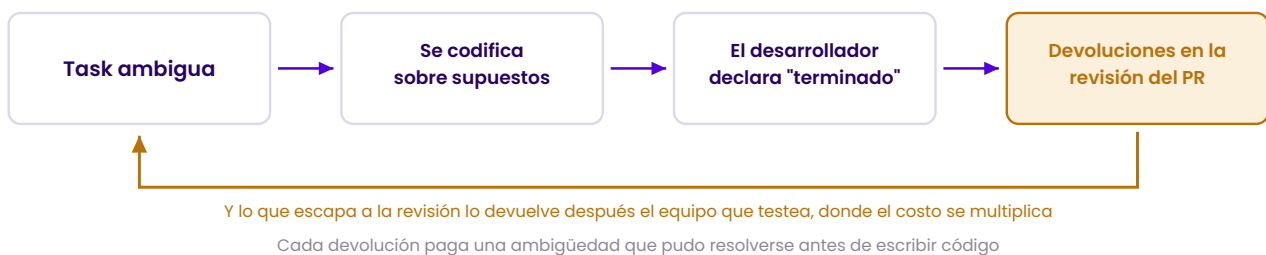
1. Resumen ejecutivo

Tech and Solve pone a consideración de Viva Aerobus esta propuesta para el proyecto **Generic API**. La intención es concreta: que la forma de diseñar, implementar, probar y entregar software deje de depender de quién tome el ticket, y pase a ser un proceso asistido por inteligencia artificial, con evidencia citada en cada paso y con puntos de control que **impiden** avanzar cuando falta algo.

Qué queremos lograr	Que la fase de desarrollo de una task — análisis, diseño, implementación, pruebas y apertura del PR — ocurra asistida por IA y con los mismos pasos siempre , de modo que todo lo verificable por máquina quede verificado antes de abrir el PR: estándar de código, cobertura de pruebas y completitud contra la task.
Qué más se gana	Además de menos devoluciones: el tiempo del revisor se libera para lo que exige criterio; el trabajo deja de vivir en la cabeza de quien lo empezó ; cada cambio queda trazable de punta a punta ; el conocimiento del sistema compone en vez de degradarse; y un integrante nuevo produce al estándar desde el arranque . Ninguno de estos depende de una promesa de acertividad — salen de cómo está construido el proceso. Los ocho, con su mecanismo, en la sección 3.
Dónde empieza y dónde termina	Empieza cuando el equipo implementador recibe una task de Jira. Termina con el PR abierto en GitHub, con la evidencia de lo desarrollado, la de las pruebas que lo cubren, y el match entre lo solicitado en la task y lo entregado, publicado como comentario del PR. La revisión del PR, el merge y el despliegue quedan fuera.
Por qué	Hoy, cuando un desarrollador declara terminado un desarrollo, empieza un ciclo de devoluciones en la revisión del PR. El estándar se aplica a mano, PR por PR; el conocimiento del sistema vive en pocas cabezas; y el costo del error se paga tarde.
Cómo	Nueve fases en orden de dependencia: primero el conocimiento , luego el estándar , luego el proceso , luego la automatización . Sobre esa base se cierran diseño, pruebas y entrega.
Bajo qué supuestos	El plan asume pruebas unitarias como alcance de automatización, y supone que los accesos, permisos y decisiones del cliente llegan en el momento en que se los necesita. El alcance de pruebas queda abierto a modificación : si el proyecto requiere integración, end-to-end u otros tipos, se incorporan revisando antes su impacto en el tiempo. Los supuestos y ese impacto están en la sección 7.
Decisión pedida	Aprobar el alcance y el orden de las fases, designar un dueño del proceso , y aceptar los cambios en la dinámica de trabajo de la sección 11.

2. El problema que resolvemos

Figura 1. El ciclo de devoluciones



El error no se origina al codificar: se origina antes, y solo se hace visible cuando alguien lo devuelve.

1. "Terminar el desarrollo" no es lo mismo que estar terminado

El desarrollador declara terminado, y ahí empieza la verdadera cola: el revisor devuelve el PR con observaciones, una y otra vez. Ese hueco entre **"ya terminé"** y **"ya pasó"** es donde vive el costo, y es el dolor que esta propuesta ataca de frente. La task envejece, el desarrollador cambia de contexto y vuelve, y el revisor gasta su tiempo — el más escaso del equipo — en observaciones que se repiten.

2. El resultado depende de quién tome la task

Dos desarrolladores con la misma task producen PRs de calidad distinta, y el mismo desarrollador produce distinto según el día y la presión de fecha. No es un problema de talento: es que nada del proceso está garantizado, así que **lo que llega a revisión varía**. Y cuando el resultado varía, planear se vuelve adivinar.

3. El alcance se descubre codificando, y los supuestos no se declaran

Las preguntas que debieron hacerse antes se terminan haciendo después, cuando cambiar de rumbo ya cuesta mucho más. Y hay algo peor que la demora: frente a una ambigüedad, alguien asume y sigue — sin que quede escrito qué asumió. El código termina construido sobre una decisión que nadie tomó de forma explícita y que nadie puede auditar después.

4. El conocimiento del sistema no está escrito

Depende de personas. El onboarding es lento, el impacto de un cambio se estima de memoria y la salida de una persona clave se siente como una pérdida de capacidad, no como un relevo.

5. El estándar técnico se aplica en el review, a mano

Las mismas observaciones se repiten PR tras PR porque nadie las verifica antes. El tiempo del revisor, que es el recurso más escaso, se consume en lo que una máquina podría validar.

6. Lo que está a medio hacer no se puede pasar a otro, y lo cerrado no deja rastro

Una task en curso vive en la cabeza y en la terminal de una persona: si se enferma, cambia de prioridad o la retoma tres semanas después, el contexto se reconstruye desde cero. Y una vez cerrada, nadie puede reconstruir qué se preguntó, qué se decidió ni sobre qué base se implementó — justo lo que hace falta cuando la task vuelve, cuando alguien toca el mismo módulo meses después, o cuando hay que explicar una decisión.

7. La devolución es la factura de todo lo anterior

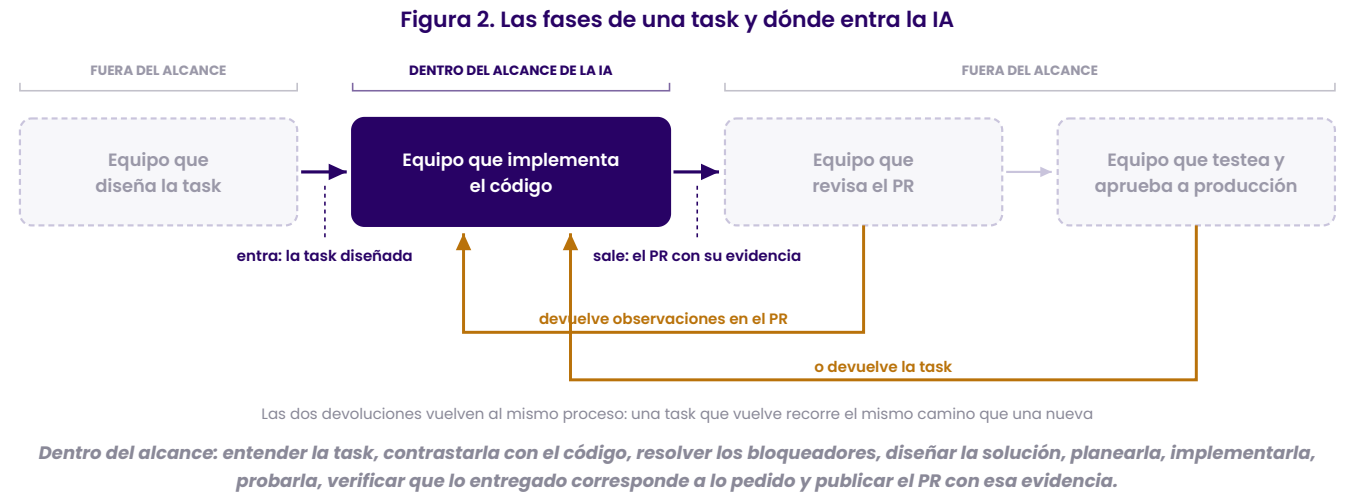
Hay dos devoluciones, y cuestan distinto. La primera y más frecuente son las **observaciones del revisor en el PR**: barata en comparación, pero repetitiva, y consume el tiempo del revisor en cosas que una máquina podría haber validado antes. La segunda es la **task devuelta por el equipo que testea**: ahí se paga de golpe todo lo que no se resolvió antes, porque hay que reabrir el contexto, reanalizar, reimplementar, volver a revisar y volver a desplegar. Juntas son la métrica más honesta del costo de la ambigüedad, y son las que esta IA busca mover.

Principio rector: el costo de corregir un error se multiplica aproximadamente diez veces por cada fase que avanza. Todo el diseño de esta IA existe para mover el hallazgo a las fases tempranas, donde corregir cuesta una conversación.

3. Qué queremos lograr

En la vida de una task intervienen cuatro equipos. Esta propuesta cubre el trabajo de uno solo — el que implementa el código — y lo cubre de punta a punta: desde entender la task hasta dejar el PR listo para revisión, con la misma disciplina cada vez. No se trata de acelerar la escritura de código.

El alcance, delimitado



- El entregable final es el PR, y debe sostenerse por sí solo: el commit con la solución, la evidencia de las pruebas que la cubren, y un comentario que enfrenta uno a uno lo solicitado en la task contra lo entregado, incluyendo lo que quedó fuera y por qué.
- La IA no reemplaza lo que hacen los otros tres equipos. Se hace responsable del tramo del medio y les entrega evidencia para que su trabajo sea más corto y sus devoluciones más raras.

En el día a día esto se traduce en algo simple: el desarrollador abre una sesión y ejecuta un solo comando. A partir de ahí, la IA conduce el ciclo y le pide a la persona exactamente lo que necesita de ella, ni más ni menos.

El objetivo concreto: quitar la varianza, no prometer perfección

- Que no haya devoluciones es imposible, y prometerlo sería la forma más rápida de perder credibilidad frente a quien revisa. Lo alcanzable es otra cosa, y es medible: que **todo lo que una máquina puede verificar quede verificado antes de que el PR exista** — siempre, con independencia de quién tomó la task y de cuánta presión de fecha había.
- Hoy la calidad de un PR es variable: depende de la persona, del día, y de si el revisor tuvo tiempo de ser exhaustivo. La IA no sube el techo — un buen desarrollador con tiempo suficiente sigue siendo mejor que cualquier automatización. Lo que hace es **subir el piso y fijarlo ahí**, con mecanismos secuenciales que se repiten sin excepción y con puntos de control que no se pueden omitir.
- No es una aspiración genérica: cada tipo de observación que hoy escribe un revisor a mano tiene un mecanismo concreto que la ataca antes de que el PR exista.

Lo que hoy encuentra el revisor a mano	Qué lo previene antes de abrir el PR
Incumplimientos del estándar de código	Las reglas normativas se aplican durante la implementación, y la auto-revisión las audita una a una antes de publicar
Faltan pruebas, o la cobertura es insuficiente	El punto de control exige suite verde y cobertura sobre lo modificado. Sin eso no hay push
Quedó algo del alcance sin hacer	La verificación de match enfrenta uno a uno lo solicitado en la task contra lo entregado, y lo publica como comentario del PR
El PR es difícil de revisar	Un solo commit limpio con la solución completa, y la evidencia adjunta
Se resolvió sobre un supuesto equivocado	El punto de control de bloqueadores lo devuelve como pregunta antes de escribir código, no después

- Qué sí y qué no.** Las devoluciones **de forma** — estándar, cobertura, completitud, legibilidad del PR — son las que se repiten y son las que la IA ataca: son verificables por máquina y no dependen de criterio. Ahí está la mayor parte del volumen.

Y la garantía llega hasta donde llega la prueba. Con pruebas unitarias se puede demostrar lo que una prueba unitaria demuestra, no más. Ampliar el alcance de la verificación exige ampliar el alcance de las pruebas, y eso está dimensionado en la sección 7.

Lo que no puede garantizar es el juicio: si el enfoque elegido es el mejor para el negocio, o si el revisor prefiere otra solución igualmente válida. Esas devoluciones van a seguir existiendo, y está bien que existan — son la conversación que aporta. **El objetivo no es un PR sin comentarios: es que el revisor no gaste su tiempo en lo que una máquina ya podía haber visto.**

Qué más queremos lograr, además de menos devoluciones

La reducción de devoluciones es el efecto más visible, pero no es el único objetivo. Estos otros no dependen de ninguna promesa de acertividad: salen directamente de cómo está construido el proceso, y son los que el resumen ejecutivo enumera de forma abreviada.

Beneficio	De dónde sale
El tiempo del revisor se libera para lo que exige criterio	Lo mecánico llega verificado y documentado en el PR, así que la revisión arranca donde antes terminaba
El trabajo deja de vivir en la cabeza de quien lo empezó	Cada fase deja artefactos versionados: una task se retoma desde cualquier máquina y cualquier sesión, sin reconstruir el contexto
Trazabilidad auditable de punta a punta	Cada línea de código se remonta a un plan, un análisis y una task, con la evidencia adjunta al PR
El conocimiento del sistema compone en vez de degradarse	Cada cierre re-documenta lo que cambió, así que el mapa mejora con el uso
El estándar deja de ser criterio personal	Está escrito, numerado y verificado por máquina; no depende de quién revise ese día
Un integrante nuevo produce al estándar desde el arranque	El estándar se aplica por construcción, no se aprende por corrección a lo largo de meses
La ambigüedad llega al equipo de diseño a tiempo	El punto de control la devuelve como pregunta antes de codificar, no después de haber gastado el esfuerzo
El equipo que testea recibe menos y mejor	Llega el match contra la task y la evidencia de pruebas, así que su trabajo arranca más arriba

Tres reglas que hacen esto confiable

- Evidencia primero.** Toda afirmación va citada contra el código. Lo que no se sabe se escribe como desconocido, nunca se inventa. Una ambigüedad es una pregunta bloqueante, no una decisión que el sistema tome por su cuenta.

Los puntos de control son mecánicos. No dependen de la buena voluntad ni de la presión de entrega: si falta un artefacto, la escritura de código o la publicación quedan denegadas.

Lo humano se marca como humano. La IA nunca declara hecho un paso que le corresponde a una persona. Lo prepara, lo entrega y lo dice explícitamente.

4. El flujo de trabajo completo, paso a paso

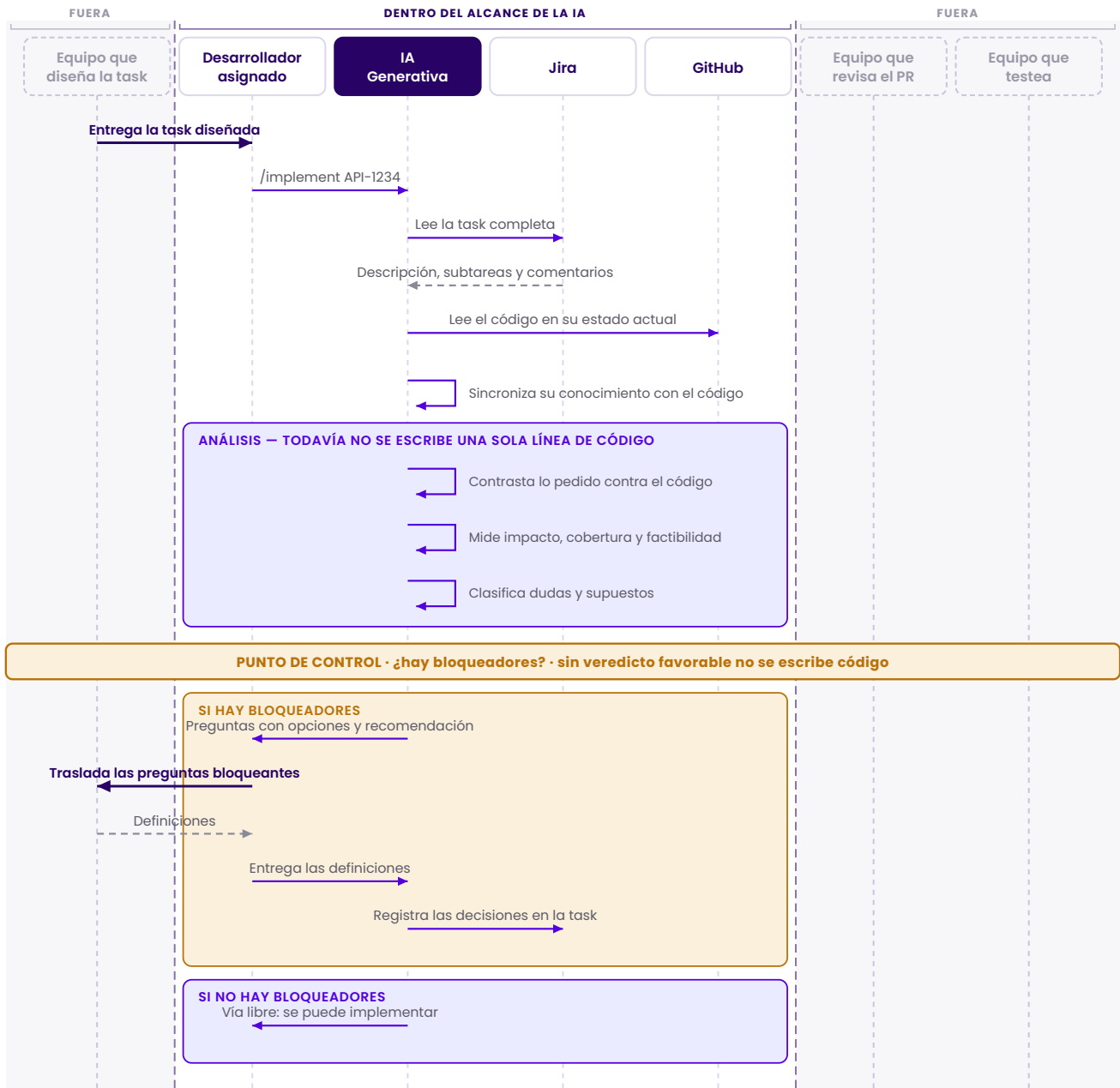
Las dos figuras siguientes muestran la secuencia real, con un carril por cada participante. Las **columnas grises de los extremos** son los otros tres equipos: quedan fuera del alcance y aparecen solo en las entregas, para que se vea con qué llegan y con qué se van.

Todo lo que pasa entre las dos líneas punteadas es lo que esta IA se hace cargo de ejecutar.

Conviene leerlas atendiendo a **quién** actúa en cada momento: la persona decide, la IA ejecuta y deja evidencia. Los **puntos de control** son las bandas ámbar que cruzan el diagrama: ahí el proceso se detiene si no se cumple una condición, y no depende de la disciplina de nadie.

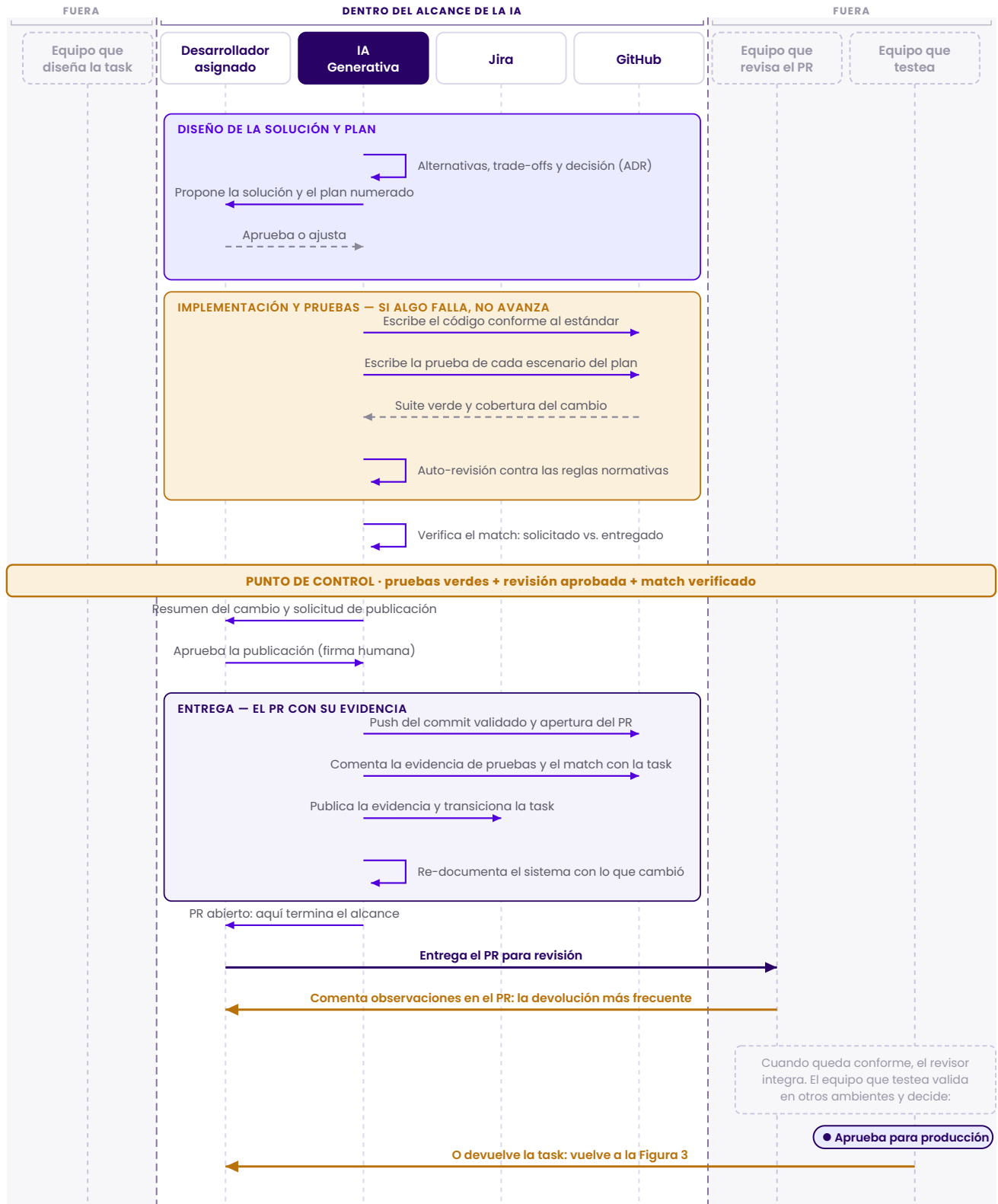
Las **flechas ámbar gruesas** de la Figura 4 son reproceso: trabajo que vuelve. Son dos caminos distintos — las observaciones del revisor en el PR y la task devuelta por el equipo que testea — y ambos reingresan al inicio de la Figura 3, porque una task que vuelve recorre el mismo proceso que una nueva.

Figura 3. Recepción de la task, análisis y punto de control de bloqueadores



El proceso arranca cuando la task diseñada llega al desarrollador. Mientras existan bloqueadores, no se escribe una sola línea de código: las preguntas vuelven al equipo que diseñó la task.

Figura 4. Diseño, implementación, pruebas y entrega del PR



Nada se publica sin pruebas verdes, revisión aprobada, match verificado y la firma explícita del desarrollador. El último acto de la IA es abrir el PR con su evidencia; de ahí en adelante deciden los otros equipos, y de sus decisiones salen las dos devoluciones.

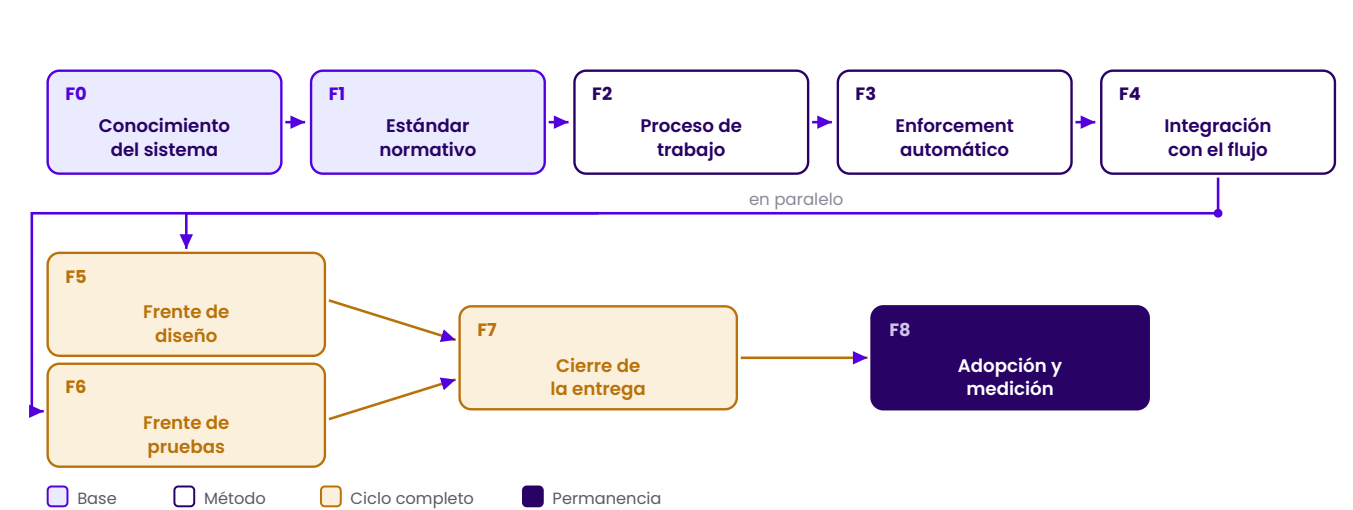
Lo que este flujo garantiza

Momento	Garantía
Al reentrar	Ni las observaciones del revisor ni una task devuelta se atienden de memoria: vuelven a pasar por el análisis y por el punto de control, con el plan y el contexto de la pasada anterior ya versionados.
Antes de codificar	Nadie escribe código sobre supuestos. La ambigüedad se devuelve como pregunta con opciones y recomendación; no se resuelve sola.
Durante	El estándar se cumple por construcción. El plan y las pruebas están numerados y trazados entre sí, uno a uno.
Antes de publicar	Suite verde, revisión aprobada y firma humana explícita. Ninguna de las tres es opcional ni se puede omitir por prisa.
Al entregar	El PR llega con la evidencia de las pruebas y con el match uno a uno contra la task. Quien revisa no tiene que reconstruir el contexto ni creer en la palabra de nadie.
Al cerrar	El conocimiento se actualiza solo, de modo que la siguiente task parte de un mapa vigente y no de uno degradado.
En cualquier punto	El trabajo está versionado: se retoma desde cualquier máquina y cualquier sesión, sin perder el contexto acumulado.

5. Las fases, y por qué en ese orden

El orden no es una preferencia de método: **cada fase es insumo de la siguiente**. Construir la automatización antes que el proceso, o el proceso antes que el conocimiento, produce trabajo que después hay que rehacer. Las fases F0 a F4 forman el camino crítico y son estrictamente secuenciales; a partir de F4 aparece el primer espacio real de paralelización.

Figura 5. Orden y dependencias de las nueve fases



F5 y F6 son las únicas fases que pueden ejecutarse en paralelo; ambas convergen en F7. F8 es la que sostiene todo lo anterior en el tiempo.

Bloque	Fases	Pregunta que responde
Base	F0, F1	¿Qué es este sistema y cómo se escribe código en él?
Método	F2, F3, F4	¿Cómo se trabaja, y cómo se garantiza que efectivamente se trabaje así?
Ciclo completo	F5, F6, F7	¿Cómo cerramos los frentes de diseño, pruebas y entrega?
Permanencia	F8	¿Cómo evitamos que todo esto se apague en seis semanas?

6. Detalle de cada fase y estimación

Base de la estimación: perfiles senior familiarizados con el dominio, a dedicación completa. Las semanas indicadas son de calendario y de orden de magnitud, pensadas para dimensionar la iniciativa; se afinan antes de comprometer fechas. F5 y F6 corren en paralelo, así que sus semanas no se suman al total.

F0 · Base de conocimiento del sistema — 4 semanas

Objetivo. Que el API deje de ser una caja negra: un mapa funcional y técnico donde cada afirmación está citada contra el código y lo desconocido está marcado como desconocido.

Entregables. Catálogo de conceptos de negocio; mapa de integraciones externas; arquitectura y patrones; mapa de dependencias entre módulos; catálogo de códigos de error; mecanismo de sincronización entre documentación y código.

Por qué va primero. Todo lo demás razona sobre esto. Un proceso excelente sobre conocimiento inexistente produce planes equivocados con mucha ceremonia. **Cómo se organiza:** el trabajo se corta por módulos, que es la unidad natural, y lo documentado se audita de forma cruzada. Esa auditoría es la que sostiene la disciplina de citar.

F1 · Estándar normativo de implementación — 2 semanas

Objetivo. Que "cómo se escribe código aquí" sea un conjunto de reglas explícitas y numeradas, y no el criterio de quien revisa ese día.

Entregables. Reglas normativas de estilo, arquitectura, robustez y proceso, extraídas del histórico real de revisiones de PR; convenciones y patrones documentados.

Por qué va aquí. Es la vara con la que se implementa (F2) y con la que se revisa (F3). Definirla después obliga a rehacer planes y revisiones ya emitidos. El trabajo se corta por familias de reglas.

F2 · Proceso de trabajo — 2 semanas

Objetivo. Que un ticket siempre recorra los mismos pasos, con criterio de salida explícito en cada uno y artefactos de nombre fijo.

Entregables. Las fases del ciclo con su criterio de salida; plantillas de cada artefacto; taxonomía de hallazgos y severidades; trazabilidad numerada entre plan y pruebas; escalamiento del rigor según riesgo (trivial, normal, riesgoso); rutas especiales para incidentes en producción.

Por qué va aquí. Es el diseño de lo que después se automatiza. Automatizar un proceso que no ha sido diseñado equivale a codificar el caos. **Cómo se organiza:** la redacción del proceso y la construcción de plantillas y taxonomías son trabajo separable, y avanzan a la vez.

F3 · Enforcement automático — 1 semana

Objetivo. Que los puntos de control no dependan de la buena voluntad.

Entregables. Guardianes que deniegan escribir código sin el análisis aprobado, y que deniegan publicar sin pruebas verdes, revisión aprobada y aprobación explícita de una persona; anclaje de la aprobación al commit exacto que fue validado; protección de las firmas humanas; suite de pruebas del propio enforcement.

Por qué va aquí. Sin esto, bajo presión de entrega el proceso se salta. Con esto, sencillamente no se puede. Los guardianes y su suite de pruebas son trabajo separable.

F4 · Integración con el flujo real de trabajo — 1 semana

Objetivo. Cero fricción de arranque: un comando, y el resto ocurre.

Entregables. Puente con Jira (lectura automática del ticket, escritura siempre detrás de aprobación); preparación automática del espacio de trabajo por ticket; progreso versionado y reanudable desde cualquier máquina; índice de trabajos por clave de ticket; varios tickets en curso sin interferencia entre sí.

Por qué va aquí. Determina si el equipo lo usa o lo evade. Un proceso correcto con un arranque costoso no se adopta.

F5 · Frente de diseño — 3 semanas

Objetivo. Que entre la task recibida y la primera línea de código exista una solución **diseñada**: alternativas evaluadas, trade-offs explícitos y decisión registrada. No cubre la definición del requerimiento — eso ocurre antes de que la task llegue.

Entregables. Fase de diseño dentro del proceso; plantilla de registro de decisión de arquitectura acotada al alcance de una task; catálogo de decisiones que se acumula task a task; criterio explícito de cuándo una task exige revisión de diseño con el tech lead.

Beneficio. Las decisiones dejan de repetirse y de discutirse dos veces. **Se ejecuta en paralelo con F6.**

F6 · Frente de pruebas — 3 semanas

Objetivo. Que la evidencia de calidad sea un resultado verificable por máquina, no un texto que alguien redactó.

Entregables. Generación asistida de pruebas a partir de la matriz de escenarios del plan; medición de cobertura sobre lo efectivamente modificado; integración con QA; criterios de aceptación funcional trazados al ticket.

Alcance de esta estimación: pruebas unitarias. Es el punto de partida, porque son las que no exigen infraestructura adicional. Incorporar integración o end-to-end cambia el esfuerzo de esta fase — el detalle está en la sección 7.

Beneficio. Menos defectos escapados, y la prueba deja de ser lo primero que se sacrifica cuando aprieta la fecha.

F7 · Cierre de la entrega: el PR como evidencia — 1 semana

Objetivo. Que el PR sea autosuficiente. Quien lo revisa debe poder juzgarlo sin reconstruir el contexto: qué se pidió, qué se hizo, con qué pruebas quedó cubierto y en qué se diferencia una cosa de la otra.

Entregables. PR con un solo commit limpio que contiene la solución completa; evidencia de las pruebas ejecutadas y de la cobertura sobre lo modificado, adjunta al PR; **comentario en el PR que enfrenta uno a uno lo solicitado en la task contra lo entregado**, con lo que quedó fuera y por qué; publicación de esa evidencia en la task de Jira y su transición de estado; re-documentación automática del sistema con lo que cambió.

Beneficio. El tiempo del revisor se dedica a juzgar la solución, no a averiguar qué se hizo. Y el conocimiento construido en F0 se mantiene vivo solo, en lugar de degradarse mes a mes. **Fuera de esta fase:** el merge, el despliegue, los runbooks de release y la observabilidad en producción.

F8 · Adopción, formación y medición — 2 semanas

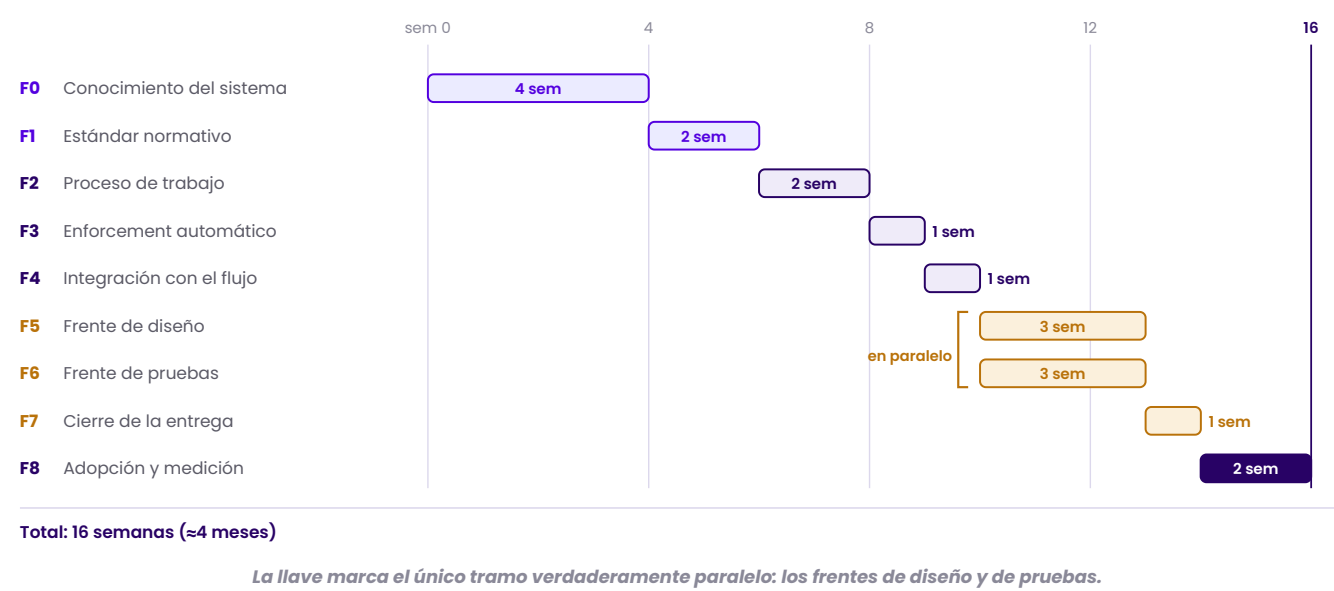
Objetivo. Pasar de "funciona con quien lo construyó" a "así trabaja el equipo".

Entregables. Secuencia de adopción por etapas (plantilla de PR, definición de listo y de terminado, matriz de impacto obligatoria, revisión de diseño, listas de verificación); formación del equipo; retrospectiva mensual del proceso; el **set de evaluación** con el que después se detecta la degradación (sección 9); y el dueño del proceso formalmente designado.

Por qué va al final y por qué no es opcional. Sin dueño y sin medición, cualquier proceso se apaga en semanas. Esta fase es la que protege toda la inversión anterior, y continúa como esfuerzo recurrente después de la semana 16.

7. Cronograma y esfuerzo

Figura 6. Cronograma del plan de trabajo



Cada fase lleva holgura reservada dentro de su duración. No es margen sobrante: absorbe coordinación, auditoría cruzada e interrupciones. Un plan al 100% de utilización no es un plan, es un deseo.

El alcance de automatización de pruebas mueve la estimación

Conviene decirlo antes de que aparezca como sorpresa: **esta estimación toma las pruebas unitarias como punto de partida**, y nada más. El esfuerzo de F6 — y con él el total — depende de qué niveles de prueba se decida automatizar, y esa decisión todavía no está tomada.

Nivel de prueba	Qué arrastra construirlo	En esta estimación
Unitarias	La suite dentro del código fuente y su ejecución determinista en local y en CI	Incluido
Integración	Datos de prueba, dobles de los servicios externos, un ambiente reproducible	No incluido
End-to-end	Ambiente completo, datos estables, orquestación y tiempos de ejecución largos	No incluido
Contrato, carga, seguridad	Herramental propio y criterios de aceptación específicos por cada tipo	No incluido

El plan está abierto a incorporarlos. Ninguno de estos niveles queda excluido por diseño: si el proyecto los requiere, se suman al alcance revisando antes su impacto en el tiempo, de modo que la decisión se tome con el costo a la vista y no después. Lo que no cambia es el mecanismo — la matriz de escenarios del plan y el punto de control de suite verde funcionan igual en cualquier nivel; lo que crece es lo que hay que construir debajo. Por eso cada nivel adicional se dimensiona cuando se decida incluirlo: su costo lo determinan los ambientes y los datos disponibles del proyecto, no la IA.

Qué puede mover estas fechas

Las 16 semanas suponen que el equipo puede avanzar sin esperar. En la práctica, una parte del plan depende de cosas que el equipo no controla: permisos, accesos y decisiones que necesitan a otras personas en la mesa. Conviene ponerlas por delante, no

descubrir las en la semana seis.

Qué se necesita del cliente	Cuándo deja de ser espera y pasa a ser bloqueo	Efecto en el calendario
Permisos y accesos: repositorio, Jira, ambientes, histórico de PR	Semana cero, antes de F0	Cada semana de demora corre el plan completo una semana
Decisión sobre el alcance de automatización de pruebas	Antes de la semana 10, cuando arranca F6	Si se define después, F6 se replanifica; si suma niveles, la estimación se recalcula
Reuniones de decisión con participantes de varias áreas	F2, F5 y F8	Agendar tres áreas suele costar más que ejecutar la decisión misma
Respuestas a preguntas bloqueantes durante el piloto	En cualquier momento	El punto de control detiene la task: el reloj corre sin que el trabajo avance
Designación del dueño del proceso	Antes de la semana 14, cuando arranca F8	Sin dueño, F8 no cierra y la inversión anterior queda sin sostener
Aprobación de licencias y cupo de uso de IA	Antes de F8	Limita el escalamiento al equipo completo
Adopción de las condiciones C-1 a C-6 (sección 11)	C-1 antes de F6; el resto durante F8	Sin C-1 no hay evidencia que automatizar y F6 queda sin sustento

Condición de validez de esta estimación. Las 16 semanas son válidas en la medida en que el cliente pueda sostener la disponibilidad que el plan supone: accesos otorgados en la semana cero, decisiones tomadas en la ventana en que se necesitan, personas disponibles para las reuniones de definición, y las condiciones de la sección 11 adoptadas en su momento. No es una salvedad de trámite: la mayor parte del riesgo de calendario de esta iniciativa no está en la construcción, está en la espera. Cuando una de estas dependencias se corre, el calendario se recalcula en ese momento — no cuando el atraso ya se acumuló.

8. Cómo sabremos que funcionó

Tres cosas, y las tres se verifican en **la primera task**: no hace falta esperar datos agregados ni levantar una línea base.

Recorre el proceso dibujado, y cada paso deja rastro

La IA no improvisa un camino distinto en cada task: recorre el flujo de las Figuras 3 y 4, en ese orden, siempre. Y **cada fase deja su propio artefacto**, con nombre fijo y versionado en el espacio de trabajo de la task: el intake, el contraste contra el código, el radio de impacto, la factibilidad, el veredicto de bloqueadores, el plan numerado, la matriz de escenarios de prueba, el resultado de la suite y la auto-revisión contra las reglas.

No es un registro que la IA deje si le sobra tiempo: **es la condición para pasar a la fase siguiente**. Y queda **consultable en cualquier momento**, por cualquiera: se puede abrir una task cerrada hace tres meses y ver qué se preguntó, qué se decidió, contra qué se contrastó, qué se planeó y qué se probó — y retomarla desde ahí sin reconstruir nada.

El criterio es simple: tomada cualquier task al azar, ese rastro está completo y se puede seguir de punta a punta.

Nada se implementa sobre decisiones sin resolver

El sistema **no escribe una línea de código hasta que toda decisión pendiente esté declarada, resuelta y aprobada por el desarrollador**. Las ambigüedades no se resuelven por cuenta propia ni quedan implícitas: se enumeran con sus opciones y una recomendación, se devuelven a quien puede definir las, y la respuesta queda registrada en la task. Recién entonces avanza.

Es el dato que más importa. Es lo que separa una herramienta confiable de una que produce volumen. Un sistema que asume obliga a auditar línea por línea para descubrir qué supuso; uno que se detiene y pregunta entrega algo de lo que se puede partir. Y no depende de la buena voluntad: el punto de control lo impide, y el desarrollador firma.

La evidencia directa: criterio por criterio, en el PR

Sabemos que funcionó cuando el PR muestra, **para cada criterio de aceptación de la task**, qué se hizo, dónde quedó, qué prueba lo cubre y con qué resultado — y cuando muestra que se implementó siguiendo las reglas designadas. No una afirmación de que está hecho: **la evidencia de que está hecho**, abrible y comprobable por quien revisa.

Qué debe mostrar el PR	Por qué eso lo hace verificable
Cada criterio de aceptación de la task, enumerado uno a uno	Ninguno queda implícito ni agrupado con otro: si la task trae seis criterios, el comentario trae seis entradas
Qué se implementó para cada criterio, y dónde quedó en el código	Se puede abrir y confirmar. No hay que creer en la palabra de nadie
Qué prueba automatizada cubre ese criterio, y su resultado	El criterio queda amarrado a una prueba, no a una opinión
Qué reglas de implementación aplicaron, y el resultado de la auto-revisión contra ellas	El cumplimiento del estándar deja de ser un juicio del revisor y pasa a ser un dato del PR
Las decisiones que se declararon y cómo se resolvieron	Queda a la vista sobre qué base se implementó, y que ninguna quedó abierta
Qué quedó fuera y por qué	Lo no hecho se declara. No se descubre en la revisión
Desviaciones respecto del plan, declaradas y aprobadas	Una desviación declarada es una decisión; una no declarada es un hallazgo

Cuándo se considera cumplido. Cuando ninguna entrada afirma "hecho" sin evidencia que se pueda abrir; cuando todo criterio de aceptación tiene al menos una prueba asociada, o una razón explícita de por qué no la tiene; y cuando las reglas aplicadas están nombradas, no supuestas.

Y lo que lo vuelve exigible, no aspiracional: el punto de control de publicación no deja abrir el PR si falta cualquiera de esas piezas. La evidencia no es un buen hábito que se pueda omitir con prisa — es la condición para publicar.

9. Lo que sigue después

Una IA generativa no se congela. El sistema cambia, el estándar se queda corto y el modelo cambia de comportamiento. Y a diferencia del software convencional, esto **no falla con un error visible: se degrada en silencio**, y se nota cuando un PR malo llega a revisión.

Lo que sigue después es **evolucionar lo que hay con lo nuevo que vaya saliendo**: incorporar las reglas que surjan de la revisión, documentar lo que quedó fuera del alcance priorizado, adoptar versiones nuevas del modelo y podar lo que resultó ruido. Para eso se destina una **capacidad dedicada** que evoluciona el sistema de IA generativa y **se adapta a la demanda de forma dinámica**: se expresa en días por mes, la prioriza el dueño del proceso y se ajusta según el volumen real de trabajo.

Una pieza la hace viable desde el primer día: el **set de evaluación** que se construye en F8 — un conjunto acotado de tasks ya resueltas que se vuelve a correr cada vez que cambia el modelo, el proceso o las reglas. Es lo que permite ver una degradación antes de que llegue a un PR real.

No es parte de esta propuesta. El alcance termina en la semana 16, con la versión 1 de la IA operando sobre tasks reales. Esta capacidad se calcula y se contrata después, cuando se conozca la demanda real.

10. Requerimientos para hacerlo realidad

Esto es lo que el plan necesita del proyecto. No es una lista de trámites: cada punto tiene un efecto concreto si falta, y están ordenados de mayor a menor impacto.

Qué requerimos	Qué pasa si no lo tenemos
Acceso al repositorio de código y a Jira, con lectura y escritura controlada	La IA pierde su puerta de entrada: sin leer la task ni el código no hay análisis posible, y el ciclo opera en modo manual
Pruebas automatizadas ejecutables con un comando, en local y en integración continua	Bloquea F6 por completo. Es la condición C-1 de la sección 11: sin evidencia que verificar, el punto de control más importante deja de existir
Disponibilidad de quien pueda responder las preguntas bloqueantes, con un plazo acordado	El punto de control detiene la task hasta que se responden: el reloj corre sin que el trabajo avance
Un dueño del proceso designado, que siga en el rol más allá de la semana 16	La IA se degrada en semanas. Es lo que F8 protege, y lo que sostiene toda la inversión anterior
Dedicación completa del equipo durante las 16 semanas	El calendario se estira de forma proporcional a la dedicación real
Acceso al histórico de revisiones de PR	F1 se construye por criterio en vez de por evidencia: el estándar sale más débil y más discutible
Tasks con una definición de listo mínima	Más ciclos de preguntas antes de empezar, y el tiempo de ciclo empeora antes de mejorar
Licencias y cupo de uso de IA para el equipo	Limita el escalamiento en F8: la IA sirve a unas pocas personas y no al equipo
Una ventana de formación de aproximadamente cuatro horas por persona	La adopción es más lenta y depende del acompañamiento uno a uno

Cada uno de estos requerimientos tiene un momento a partir del cual deja de ser una espera y pasa a ser un bloqueo con costo en calendario. Ese detalle, con su efecto concreto sobre las fechas, está en la sección 7.

11. Cambios que requiere la dinámica de trabajo

La IA no funciona sobre cualquier forma de trabajar. Hay prácticas del proyecto que **deben cambiar** para que pueda garantizar lo que promete, y no son preferencias metodológicas: son condiciones técnicas de las que depende que la evidencia sea verificable por máquina y no la afirmación de alguien.

Cada una queda marcada con un identificador, para poder seguirla como tema aparte con su propio responsable y su propia fecha.

C-1 · Las pruebas automatizadas deben ser obligatorias dentro del código fuente

Por qué la IA lo necesita: el punto de control de publicación exige suite verde como evidencia verificable. Si las pruebas no viven en el repositorio y no se ejecutan con un comando, no hay nada que la IA pueda verificar ni adjuntar al PR. **Si no cambia:** la evidencia de calidad vuelve a ser una afirmación humana, el punto de control más importante deja de existir y el PR autosuficiente se cae. **Es la única de estas condiciones que debe estar resuelta antes de que F6 arranque.**

C-2 · La task debe cumplir una definición de listo mínima

Por qué: el análisis contrasta lo pedido contra el código. Sin un enunciado analizable, el punto de control bloquea y la task rebota antes de empezar. **Si no cambia:** más ciclos de preguntas al inicio, y el tiempo de ciclo empeora antes de mejorar.

C-3 · Las decisiones se escriben en la task, no solo se conversan

Por qué: la task es la memoria del proyecto. La IA la lee al arrancar, y una task devuelta se retoma desde ahí. **Si no cambia:** cada reproceso vuelve a discutir lo que ya se había decidido.

C-4 · El PR lleva un solo commit limpio y la evidencia como comentario

Por qué: es el formato del entregable, y es lo que ancla la aprobación al commit exacto que fue validado. **Si no cambia:** hay que rediseñar el cierre de la entrega y se pierde la trazabilidad entre lo aprobado y lo publicado.

C-5 · Las preguntas bloqueantes tienen responsable y plazo acordado

Por qué: el punto de control detiene la task hasta que se responden. Es el comportamiento correcto, pero sin un acuerdo de respuesta se vuelve un cuello de botella. **Si no cambia:** tasks bloqueadas por tiempo indefinido, y el equipo aprende a rodear el proceso.

C-6 · El equipo que testea registra la causa de cada devolución

Por qué: cada devolución que la IA debió prevenir es una mejora concreta para la IA — una regla que falta, un hueco de conocimiento, un punto de control ausente. Sin la causa registrada, la devolución no se puede convertir en nada. **Si no cambia:** la IA deja de aprender de sus propios errores y la evolución de la sección 9 se queda sin insumo.

Estas condiciones no se imponen el primer día: F8 las introduce por etapas y cada una necesita un dueño identificable. La excepción es C-1, que es una precondition y no una meta de adopción: sin pruebas automatizadas en el código fuente no hay nada que automatizar, y F6 no tiene sobre qué operar.

12. Riesgos y mitigación

Riesgo	Mitigación
Los permisos y accesos llegan tarde	Se piden y se verifican en la semana cero, antes de comprometer fechas. El plan arranca por lo que no requiere acceso, y cada semana de demora se traslada explícitamente al calendario en vez de absorberse en silencio.
Las decisiones que involucran a varias áreas se demoran	Cada decisión pendiente lleva fecha límite y una opción por defecto documentada , que se aplica si no hay respuesta. Toda decisión tomada por defecto queda registrada como supuesto, visible y reversible.
El alcance de automatización de pruebas se decide tarde, o crece	Se decide antes de la semana 10. Si crece por encima de las unitarias, se recalcula y se cotiza antes de continuar, no después. La sección 7 deja escrito qué arrastra cada nivel.
La disponibilidad para reuniones de decisión es menor que la supuesta	Las decisiones se preparan por escrito con opciones y recomendación, para que la reunión sirva para decidir y no para explicar. Lo que se puede resolver de forma asíncrona no se agenda.
Las condiciones C-1 a C-6 no se adoptan	Cada una tiene responsable y fecha desde el arranque. C-1 es precondition de F6: si no está lista, F6 no arranca y se replanifica antes de gastar el esfuerzo.
F0 se desborda porque el sistema es más grande de lo estimado	Se prioriza por los módulos de mayor tránsito y el resto se documenta bajo demanda. Todo lo no cubierto queda marcado de forma explícita como desconocido, de modo que la IA lo trate como pregunta bloqueante y no como dato. El hueco se ve; no se disimula.
La IA se degrada después de la entrega	La capacidad dedicada de la sección 9 existe para eso, y el set de evaluación hace visible la degradación antes de que llegue a un PR.
La salida de un integrante detiene el proyecto	Auditoría cruzada desde F0: cada frente es revisado por alguien distinto de quien lo produjo, así que nadie es el único que entiende su parte.
Se percibe como burocracia y el equipo lo rechaza	El rigor escala al riesgo: un cambio trivial no lleva registro de decisión ni revisión de diseño. La adopción es por etapas (F8), no "todo el lunes".
El conocimiento queda obsoleto y la IA razona sobre datos viejos	La sincronización es obligatoria y verificada; F7 hace que cada cierre re-documente el sistema.
La IA afirma lo que no sabe	Regla no negociable de evidencia: todo citado contra el código, lo desconocido marcado como tal, y la ambigüedad convertida en pregunta bloqueante.
Se salta un punto de control por presión de entrega	Los puntos de control deniegan la operación (F3). No dependen de disciplina individual.
El enforcement genera falsos positivos y frena el trabajo	Calibración con tickets reales antes de escalar al equipo completo.
La formación de F8 no alcanza para la adopción	El acompañamiento se hace sobre trabajo real en lugar de en aula, y la retrospectiva mensual detecta temprano si el equipo no está usando la IA.

Documento vivo. El alcance de las fases, los criterios de verificación de la sección 8 y las estimaciones se afinan con el equipo antes de comprometer fechas. Las cifras de esfuerzo dimensionan la iniciativa; el precio se cotiza una vez aprobado el alcance. Las 16 semanas son válidas bajo los supuestos declarados en la sección 7 — alcance de pruebas unitarias, y disponibilidad del cliente para accesos, decisiones y definiciones en la ventana en que el plan los necesita.